# Embedded Artificial Intelligence in Battery Management Systems: Pruning and Quantization for Efficient State-of-Charge and State-of-Health Estimation

Anonymous

*Anonymous,*

**Abstract**

State-of-charge (SOC) and state-of-health (SOH) estimators based on recurrent neural networks can achieve high accuracy, but their computational and memory cost on embedded hardware is often insufficiently characterised. This work quantifies the accuracy, timing, and resource trade-offs of Long Short-Term Memory (LSTM) and Multi-Layer Perceptron (MLP) based SOC/SOH estimators deployed on an STM32H753ZI microcontroller. Using the open *LFP DoE Cycle Ageing Dataset*, we train full-precision baseline models and obtain compressed variants via pruning and weight quantization. In long-horizon streaming replay on the test set the SOC estimators attain mean-absolute errors (MAE) between 2.3 % and 2.8 % of full scale, while the SOH estimators reach MAE values between 0.9 % and 1.5 %. Hardware-in-the-loop benchmarks on the STM32 show that pruning reduces both the flash footprint and the energy per inference by roughly 40 % for SOC and about 45 % for SOH, while slightly improving SOC MAE and keeping SOH MAE within a few tenths of a percent of the dense baseline. Quantization further shrinks the flash footprint to around half of the baseline size for both tasks and keeps the MAE within a few tenths of a percent of the corresponding full-precision models, at the expense of increased inference energy, especially for SOC. The presented methodology and measurements provide a reproducible reference for implementing and comparing data-driven battery state estimators on resource-constrained Battery Management System (BMS) hardware.

*Preprint submitted to Engineering Applications of Artificial Intelligence*

*Keywords:* State of charge, State of health, LSTM, Pruning, Quantization, STM32, Battery diagnostics, Embedded AI

---

## Nomenclature

*Acronyms*

| | |
|---|---|
| **BMS** | Battery management system |
| **SOC** | State of charge |
| **SOH** | State of health |
| **DoE** | Design of Experiments |
| **LFP** | Lithium iron phosphate |
| **LSTM** | Long short-term memory network |
| **MLP** | Multi-layer perceptron |
| **STM32** | 32-bit ARM Cortex-M microcontroller family |
| **MAE** | Mean absolute error |
| **RMSE** | Root-mean-square error |
| **INT8** | Signed 8-bit integer representation (quantized weights) |
| **FP32** | 32-bit floating-point representation |

*Symbols*

| | |
|---|---|
| $t$ | Time |
| $U(t)$ | Terminal voltage at time $t$ |
| $I(t)$ | Current at time $t$ (positive for discharge) |
| $T(t)$ | Cell temperature at time $t$ |
| $\mathrm{EFC}(t)$ | Equivalent full cycles accumulated up to $t$ |
| $Q_c(t)$ | Cumulative charge throughput up to $t$ |
| $C_{\mathrm{nom}}$ | Nominal discharge capacity of the cell |
| $C_{\mathrm{app}}$ | Apparent capacity inferred from the latest check-up |

| | |
|---|---|
| DoD | Depth of discharge |
| $\mathrm{SOC}(t)$ | State of charge at time $t$ |
| $\mathrm{SOH}_k$ | State of health at reference cycle $k$ |
| $\mathbf{x}_t$ | Input feature vector at time $t$ |
| $\mathbf{h}_t$ | LSTM hidden state at time $t$ |
| $\mathbf{c}_t$ | LSTM cell state at time $t$ |
| $\mathbf{z}_t$ | Concatenation of input and previous hidden state at time $t$ |
| $\mathbf{i}_t, \mathbf{f}_t, \mathbf{o}_t, \mathbf{g}_t$ | Input, forget, output, and candidate gates at time $t$ |
| $\sigma(\cdot)$ | Logistic sigmoid activation function |
| $\tanh(\cdot)$ | Hyperbolic tangent activation function |
| $\odot$ | Element-wise (Hadamard) product |
| $W$ | Weight matrix (LSTM or MLP) |
| $b$ | Bias vector |
| $q$ | Quantized weight (INT8 code) |
| $s$ | Per-row quantization scale (FP32) |
| $\hat{w}$ | Reconstructed floating-point weight after dequantization |
| $\mathcal{Q}_s(\cdot)$ | Uniform mid-tread quantizer with scale $s$ |
| $W_{\mathrm{ih}}^{\mathrm{int8}}$ | Quantized input-to-gate weight matrix |
| $W_{\mathrm{hh}}^{\mathrm{int8}}$ | Quantized hidden-to-gate weight matrix |
| $s_{\mathrm{ih}}, s_{\mathrm{hh}}$ | Per-row quantization scale vectors for $W_{\mathrm{ih}}^{\mathrm{int8}}$ and $W_{\mathrm{hh}}^{\mathrm{int8}}$ |
| $b_{\mathrm{fp32}}$ | Floating-point bias of a given gate |
| $g$ | Gate pre-activation used in the quantized LSTM kernel |

## 1. Introduction

Accurate online estimation of the state of charge (SOC) and the state of health (SOH) remains a central challenge for battery powered systems [1, 2, 3, 4]. Classical model-based estimation schemes often struggle to track long-term

degradation under highly dynamic current and temperature conditions. Neural networks can capture such effects and have recently been surveyed comprehensively for SOC estimation in [1], yet the majority of published solutions remain purely theoretical: they report accuracy on offline datasets but rarely demonstrate how the models behave once deployed on embedded targets with tight memory and energy budgets. This work closes this gap by optimising SOC and SOH Long Short-Term Memory (LSTM) models for microcontroller deployment using pruning and quantization. The methodology is motivated by modular stationary storage systems in remote microgrids, where battery storage modules must operate autonomously without reliable infrastructure. In such settings, the battery management system must compute SOC and SOH within the flash, RAM, and energy limits of an STM32H753ZI-class microcontroller. Similar demands arise in mobile applications where devices must run for long periods without cloud support. Translating high-fidelity neural estimators into this environment is therefore an enabling step for future resource-aware Battery Management System (BMS) platforms. While many machine-learning models for state estimation have been proposed [1], accompanying artefacts such as scaling statistics, hidden-state handling, and quantization parameters are rarely documented in sufficient detail to enable faithful reproduction on microcontrollers. Even fewer works report resource usage, energy consumption, or long-horizon streaming behaviour of the deployed models, with only a handful of studies analysing Tiny Machine Learning (TinyML)-style SOC or SOH estimators on microcontrollers [10, 11, 12]. Against this backdrop, we focus on closing the gap between theoretical validation and embedded deployment, with a particular emphasis on optimisation for memory, runtime, and energy. In the remainder of this work we rely on the resulting Lithium Iron Phosphate (LFP) DoE Cycle Ageing Dataset [13]. The data capture long-term cyclic ageing of numerous LFP cells under full and fractional factorial conditions with controlled temperature, charge and discharge rates, depth-of-discharge variations, and interleaved check-ups containing capacity tests, OCV sweeps, and hybrid pulse-power characterisation. This breadth provides a realistic basis for training and evaluating

SOC and SOH estimators across a wide range of ageing patterns.

## 2. Related Work and Motivation

### *2.1. SOC/SOH Estimation*

Existing SOC/SOH estimation methods can be grouped into three families [2, 3, 4, 14]. First, *measurement-derived* techniques use Coulomb counting and open-circuit-voltage (OCV) look-up tables during rest periods. They provide inexpensive proxies but suffer from drift, offset accumulation, and a strong dependence on initialisation and periodic recalibration. Second, *model-based* observers combine equivalent-circuit or electrochemical models with Kalman filtering and its variants. These approaches can offer physical interpretability but require careful parameter identification and are sensitive to model mismatch and ageing-induced parameter drift. Third, *data-driven* methods employ machine-learning regressors, including LSTM networks, Transformers, and hybrid structures, to map sequences of current, voltage, and temperature to SOC and SOH estimates. Key challenges for such models are robustness when operated under conditions that differ from the training data and the careful treatment of normalisation and state handling on embedded targets. Across these families the reported accuracy spans a wide range. Electrochemical or Equivalent Circuit Model (ECM)-based observers with carefully tuned filters often achieve SOC root-mean-square errors (RMSE) well below 1 % of full scale and sub-percent SOH capacity errors on laboratory test cycles [15, 16, 17, 18]. Recent sequence-model approaches based on N-BEATS and Bidirectional LSTMs (BiLSTMs) for sodium-ion cells report SOC Mean Absolute Error (MAE) between roughly 0.1 and 0.4 % [19, 20], whereas TinyML-style SOC estimators deployed on microcontrollers typically operate in the 1–3 % error regime [10, 21, 11]. These results provide a useful reference band for the SOC/SOH errors obtained in this work. Kalman filters, particle filters, and equivalent-circuit-model observers therefore dominate the classical literature, yet their accuracy can degrade over long operating periods with pronounced current and temperature gradients [16, 18].

Recurrent neural networks, and in particular LSTM-based regressors, address these limitations by integrating long temporal context and absorbing residual modelling errors [22, 18]. In this work we deliberately restrict the architecture search to a single-layer LSTM with a fully connected Multi-Layer Perceptron (MLP) output layer. LSTMs retain explicit cell states with separate input, forget, and output gates, which made it straightforward to hand-code a numerically stable, stateful kernel in C and to reuse the same backbone for both SOC and SOH. Gated recurrent units (GRUs) or Transformer variants could offer alternative trade-offs between accuracy and cost, but exploring multiple architectures would dilute the focus on the embedded tool flow and is left as future work. Most publications on LSTM-based estimators maximise accuracy with complex architectures and rarely examine how the resulting models behave once deployed on constrained controllers. Even within the LSTM family, sliding-window inference is commonly employed because it delivers high accuracy, but it also requires substantial RAM to buffer multiple past samples. A stateful LSTM that hands over its hidden and cell vectors step by step retains the temporal information without maintaining large windows and therefore forms the first optimisation step in our architecture. The associated handling of hidden states, feature scaling, and numerical stability is rarely described in the literature, which is why this work explicitly targets embedded SOC/SOH deployment.

### *2.2. Embedded Deployment Strategies*

Commercial toolchains such as ONNX Runtime, TensorRT, and STM32Cube.AI can export many feed-forward networks [23, 24, 25], but they fall short for stateful SOC/SOH estimators that must maintain continuous internal states between time steps. In addition, safety-critical deployments require predictable behaviour and source code that can be audited, which is difficult when quantization and optimisation are hidden inside proprietary binaries. Besides the stateful execution, embedded operation also demands explicit control over footprint, hence pruning and quantization must be applied in a transparent way rather than via opaque compiler passes. Complementary work such as [26]

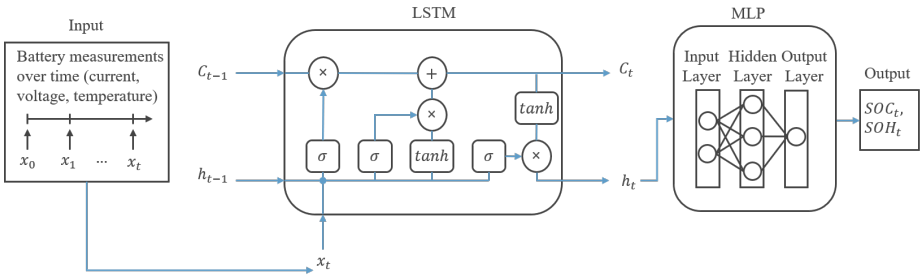


Figure 1: Schematic of the deployed SOC/SOH estimator. Preprocessed feature streams are fed into a single-layer LSTM whose hidden state is propagated across time; at each step a lightweight MLP head maps the hidden representation either to SOC or SOH, enabling stateful streaming inference on the STM32.

quantifies execution time and memory for a variety of SOC algorithms on microcontrollers, while PQ Bench [27] provides a systematic benchmark of modern pruning and quantization techniques and demonstrates that they can deliver strong compression–accuracy trade-offs across diverse deep-learning workloads. Our study builds on these insights by applying pruning and quantization to stateful LSTM + MLP SOC/SOH estimators and by extending the analysis to a full STM32 firmware implementation with detailed measurements of flash, RAM, latency, and energy in a realistic BMS setting. We therefore opted for a manual path: extracting PyTorch tensors, pruning rows and columns according to saliency, applying per-row symmetric quantization for the LSTM weights, and re-implementing the gates and activations in plain C. This guarantees that every multiplication, scaling, and bias term seen on the STM32 matches the reference simulation while keeping the model within the available RAM and flash budgets.

## 3. Dataset and Preprocessing

### *3.1. Ageing Experiments and Load Profiles*

All experiments rely on the high-resolution LFP DoE Cycle Ageing Dataset [13]. The measurements originate from a design-of-experiments study on commer-

cially available JGCFR18650-1800mAh-3.2V LFP cells [28] conducted in a university laboratory. Cells were cycled inside a climatic chamber at 25°C while the following factors were systematically varied:

$$C_{\mathrm{chg}} \in [0.5\,\mathrm{C},\, 0.9\,\mathrm{C}], \tag{1}$$

$$C_{\mathrm{dis}} \in [1.0\,\mathrm{C},\, 2.5\,\mathrm{C}], \tag{2}$$

$$\mathrm{DoD} \in [45\%,\, 65\%]. \tag{3}$$

A mixed full/fractional factorial plan assigned fifteen test cells to eight operating points, each condition being executed in duplicate to ensure reproducibility. Between ageing loops, dedicated check-ups consisting of capacity tests, open-circuit-voltage sweeps, and hybrid pulse power characterisation were inserted to parameterise the cells. In addition to the strictly controlled cycling sequences, stationary-storage profiles were injected to emulate realistic power flows [29]. These profiles are derived from residential photovoltaic (PV) home storage applications, which are characterised by stochastic load and generation patterns that stress the cell dynamics differently than constant-current cycling. The raw data contain uniformly sampled (1 Hz) measurements of voltage, current, terminal temperature, and auxiliary channels and are stored as partitioned Parquet tables for efficient retrieval. The dataset is published in [13]; the cube notation follows standard DoE conventions as summarised in [30]. Together, Figures 3 and 4 illustrate the diversity of SOC transients and ageing behaviour that the estimators must capture: short-term SOC dynamics are driven by fast current swings and mode changes between profiles, whereas SOH evolves slowly over hundreds of days and depends on the cumulative operating conditions of each cell.

### *3.2. Feature Engineering*

The SOC model consumes the six-dimensional feature vector

$$\mathbf{x}_t^{\mathrm{SOC}} = \left[U(t),\, I(t),\, T(t),\, Q_c(t),\, \tfrac{\mathrm{d}U}{\mathrm{d}t}(t),\, \tfrac{\mathrm{d}I}{\mathrm{d}t}(t)\right], \tag{4}$$

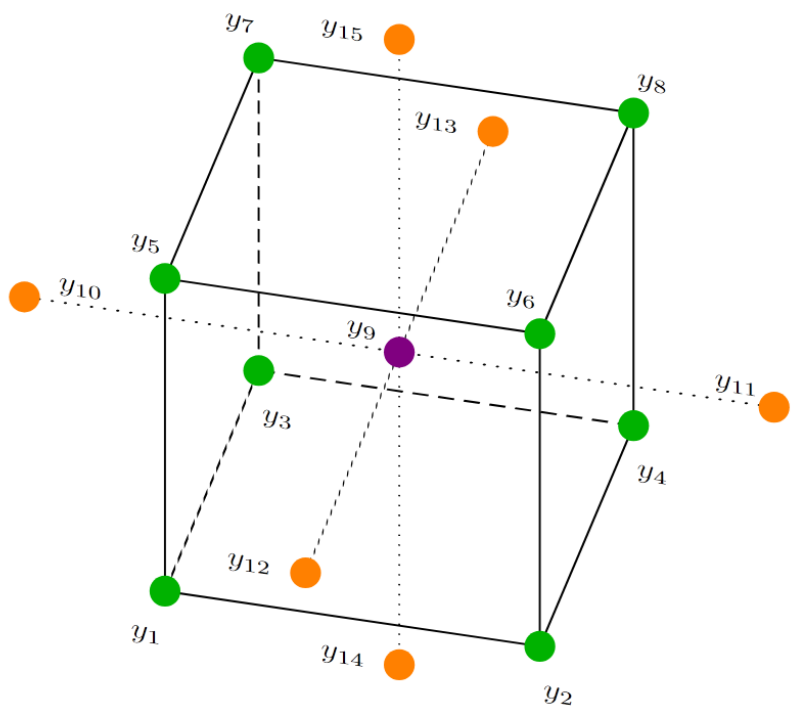


Figure 2: Design-of-experiments cube for the LFP DoE Cycle Ageing Dataset. The three axes correspond to charge C-rate, discharge C-rate, and depth of discharge (DoD); the highlighted vertices indicate the eight factorial operating points assigned to the test cells.

where temporal derivatives are obtained via centred finite differences. Each component is normalised using a robust scaling operation based on the median $m$ and interquartile range $r$:

$$\tilde{x}_{t,k}^{\mathrm{SOC}} = \frac{x_{t,k}^{\mathrm{SOC}} - m_k}{r_k + \varepsilon}, \qquad \varepsilon = 10^{-6}. \tag{5}$$

The same $(m_k, r_k)$ pairs are embedded in the firmware so that the STM32 receives identically normalised inputs. The SOH model uses a slightly different feature stack to capture slow-varying degradation markers:

$$\mathbf{x}_t^{\mathrm{SOH}} = \big[t,\, U(t),\, I(t),\, T(t),\, \mathrm{EFC}(t),\, Q_c(t)\big], \tag{6}$$

where $t$ is the cumulative time stamp and $\mathrm{EFC}(t)$ denotes the equivalent full cycles accrued up to sample $t$. These channels are likewise robustly normalised,

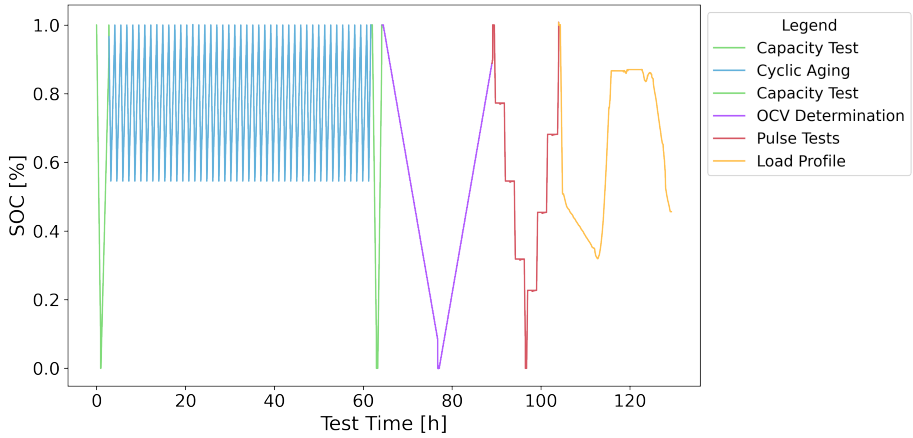


Figure 3: Representative SOC trajectory illustrating selected segments of the experimental protocol used in this study. The plot shows a portion of the cyclic ageing procedure with defined charge and discharge currents and a specified depth-of-discharge (DoD), followed by the diagnostic checkup sequence comprising OCV determination, HPPC pulses, and capacity tests. In addition, the trajectory includes a load profile taken from a stationary battery storage system. Together, these segments capture both controlled laboratory diagnostics and application-oriented operating conditions.

relying on median and interquartile range rather than mean and variance to reduce the influence of outliers in current and voltage measurements [31, 32]. During deployment the SOC and SOH models run independently on the microcontroller: the SOC and SOH model provides a high-rate estimate at 1 Hz. In principle both estimates could be fused more tightly, but in this work we keep the models separate in order to study their resource usage and accuracy in isolation.

### *3.3. Reference SOC and SOH targets*

The ground-truth SOC trajectory is derived from Coulomb counting with drift compensation. Denoting the apparent cell capacity after a given check-up by $C_{\mathrm{app}}$, the SOC inside a charge-discharge segment is computed as

$$\mathrm{SOC}(t) = \mathrm{SOC}(t_0) - \frac{1}{C_{\mathrm{app}}} \int_{t_0}^{t} I(\tau)\,\mathrm{d}\tau, \tag{7}$$

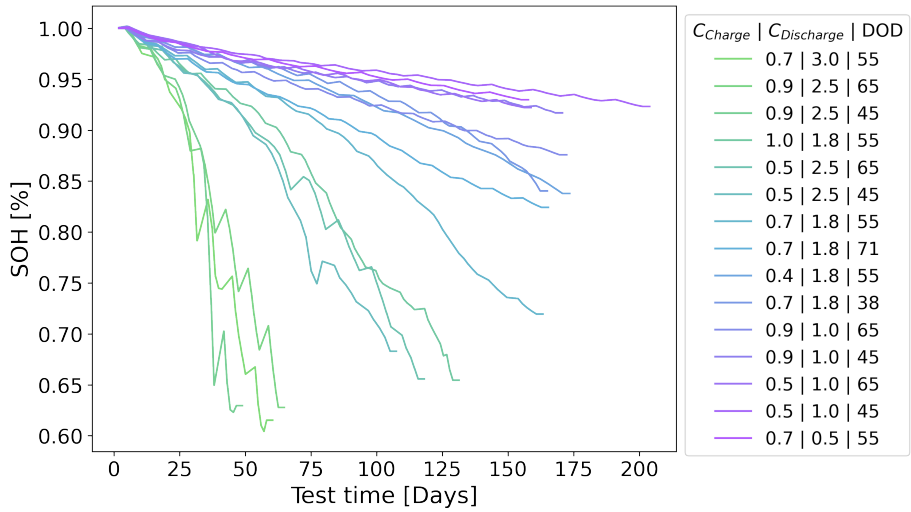


Figure 4: Evolution of the state of health across all LFP cells in the ageing campaign, shown over the total experimental duration. The spread between trajectories reflects the different operating points in the factorial plan (C-rates, depth of discharge, profile mix) and motivates the use of a data-driven SOH estimator that generalises across cells and loading conditions.

where the integral is reset to SOC = 1 after each constant-voltage (CV) termination. The reset mitigates the bias that would otherwise accumulate due to capacity fade, yet the long-term traces still reveal gradual changes in the SOC envelope as $C_{\text{app}}$ decreases. State of health is obtained from the periodic capacity check-ups through

$$\text{SOH}_k = \frac{C_k}{C_{\text{nom}}}, \tag{8}$$

with $C_k$ representing the measured discharge capacity during the $k$-th reference cycle (comprising capacity test, open-circuit-voltage sweep, and hybrid pulse-power characterisation) relative to the nominal capacity $C_{\text{nom}}$. These targets highlight the need for a data-driven estimator that remains accurate despite the evolving capacity landscape, which motivates the use of recurrent neural networks for both SOC and SOH predictions. The reference SOC and SOH traces are therefore offline constructs. They rely on information that is not available in deployment, such as long rest periods for OCV stabilisation and

capacity from periodic full cycles, and they assume perfect initialisation and recalibration. These constructions follow common practice in battery management, where SOC is obtained by Coulomb counting and SOH is expressed as the ratio between present and nominal capacity [33, 2, 3]. On the STM32 the estimators operate purely on the streamed measurements and their internal states; the reference trajectories only serve as ground truth for training and evaluation.

### *3.4. Streaming inference setup*

Because the SOC and SOH estimators are stateful, the LSTM hidden and cell states are continuously forwarded from one sample to the next along the full input stream. At each sampling instant the current feature vector is processed together with the previous hidden and cell states, which yields the new states and the corresponding SOC or SOH estimate. We therefore operate the models in a purely step-by-step streaming regime without forming sliding input windows. On the host the full normalised sequence from the LFP DoE Cycle Ageing Dataset is processed once in this fashion, and the resulting SOC and SOH estimates are aligned with the reference trajectories. On the STM32 firmware the same step-wise LSTM implementation is used, driven by the UART feature stream, so that host and embedded benchmarks reflect identical inference conditions. The streaming dashboards in Figs. 8 and 11 illustrate a complete run for one representative cell.

## 4. Model Architecture and Baseline Performance

### *4.1. LSTM architecture in PyTorch*

Both SOC and SOH estimators employ the same recurrent equation set but differ in size according to their training releases. The SOC configuration uses $H = 64$ hidden units and an MLP output layer of width 64, whereas the SOH configuration increases both dimensions to 128 to cope with the longer temporal context. LSTM gates $i_t, f_t, o_t$ and candidate state $g_t$ follow the standard

formulation used in modern RNN surveys [34]. All models are implemented in PyTorch [35].

$$
\begin{aligned}
\mathbf{z}_t &= [\mathbf{x}_t, \mathbf{h}_{t-1}], \\
\mathbf{i}_t &= \sigma(W_i \mathbf{z}_t + \mathbf{b}_i), \quad \mathbf{f}_t = \sigma(W_f \mathbf{z}_t + \mathbf{b}_f), \\
\mathbf{o}_t &= \sigma(W_o \mathbf{z}_t + \mathbf{b}_o), \quad \mathbf{g}_t = \tanh(W_g \mathbf{z}_t + \mathbf{b}_g), \\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t, \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t).
\end{aligned} \tag{9}
$$

Both SOC and SOH models are trained with a mean-squared-error loss between the prediction and the corresponding reference at each training sample. During validation we additionally report mean absolute error (MAE), root-mean-square error (RMSE), and $R^2$. The trained parameters, together with the corresponding configuration files and optimiser statistics, are archived to enable deterministic replays of both models.

### *4.2. Training Notes*

Training is carried out on a GPU node equipped with NVIDIA A100-class accelerators using mixed-precision (FP16) arithmetic. Both SOC and SOH models use the AdamW optimiser with cosine-annealing warm-restart scheduling, gradient clipping, and mini-batch training [36, 37, 38, 39]. For SOC we use a learning rate of $1.5 \times 10^{-4}$, a batch size of 512, and sequence chunks of length 2024, whereas SOH training employs a learning rate of $2.0 \times 10^{-4}$, a batch size of 128, and chunks of length 2048 with otherwise identical optimisation settings. Although deployment operates the LSTM in fully stateful, step-by-step streaming mode, training thus relies on truncated backpropagation through time over these windows. Within each chunk the hidden and cell states are propagated across time exactly as in deployment, so the stateful update mechanism is already learned during training, while the windowing allows randomised sampling of sequence segments, accelerates convergence, and improves generalisation across the diverse ageing trajectories.

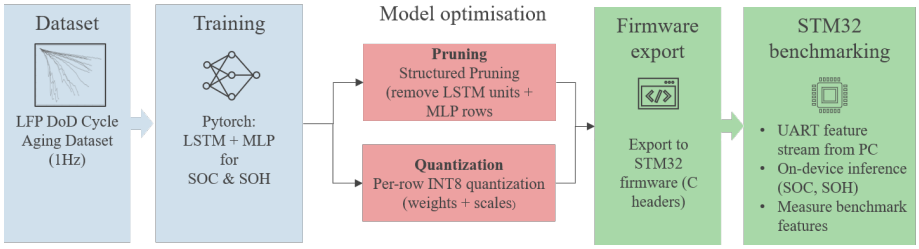


Figure 5: End-to-end pipeline from dataset to embedded benchmarking. The LFP DoE Cycle Ageing Dataset feeds PyTorch training on a GPU workstation; the resulting SOC and SOH LSTM + MLP models are pruned and quantized, exported as C headers, and deployed on an STM32H753ZI board, where flash/RAM usage, inference time, latency, and energy are measured under streaming replay.

### *4.3. Baseline Accuracy*

On the workstation the SOC and SOH base models are first evaluated in streaming mode on the full ageing trajectory of one representative LFP cell from the LFP DoE Cycle Ageing Dataset [13], yielding baseline MAE and RMSE values for both tasks. These Python replays operate on the original PyTorch implementations and serve as a numerical reference for all subsequent experiments. The same checkpoints are then pruned, quantized, and exported to STM32 firmware, where identical feature streams are replayed to measure not only accuracy but also flash and RAM usage, system latency, on-device inference time, and energy per inference (cf. Section 7). Figure 5 summarises this training-to-firmware benchmarking pipeline.

## 5. Pruning

Pruning reduces the size and computational cost of a neural network by eliminating parameters or whole units that contribute little to the overall mapping, while keeping the input-output behaviour as close as possible to the dense baseline [40, 41]. In our setting we apply structured pruning at the level of complete LSTM hidden units so that the resulting model remains well suited for efficient C implementations on the microcontroller.

### 5.1. Structured magnitude pruning

Structured magnitude pruning removes the lowest-saliency LSTM units until only 45 hidden channels remain in the SOC model (30% sparsity), with analogous settings for the SOH experiments. Reducing the state dimension is a prerequisite for fitting the estimators into the flash and SRAM budgets of the target STM32-based BMS and for shortening the on-device inference time, inspired by structured sparsity approaches for deep and recurrent networks [42, 43, 44]. In analogy to the group-lasso style regularisers proposed for LSTMs in [42, 43], which aggregate weights across gates via L2 norms, we define a saliency score for each hidden unit $h$ that combines all incoming weights from the different gates:

$$s_h = \sum_{g \in \{i,f,o,g\}} \left( \|W_{\mathrm{ih}}^{(g)}[h,:]\|_2 + \|W_{\mathrm{hh}}^{(g)}[h,:]\|_2 \right). \tag{10}$$

Units are ranked by $s_h$ and the top $H'$ indices are retained. In the final deployments we use $H' = 45$ hidden units for the SOC estimator and $H' = 90$ units for the SOH estimator, corresponding to roughly 30 % channel-wise sparsity with respect to the original architectures. Pruned rows and columns are removed jointly from all gate matrices as well as from the downstream MLP weights, ensuring that the reduced network still implements a valid recurrent mapping. After each pruning step the network is fine-tuned on the training data before the next pruning round commences.

### 5.2. Resulting pruned models

The resulting pruned SOC and SOH models still run in FP32 but reduce flash consumption and runtime considerably once compiled for STM32. Section 7 summarises the corresponding latency and memory savings. These pruned variants are evaluated alongside their base and quantized counterparts throughout the results section.

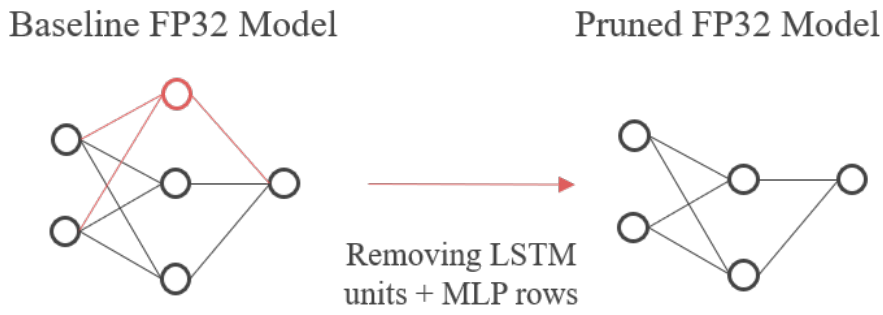


Figure 6: Schematic illustration of structured pruning. The left panel highlights a low-saliency hidden unit and its incident connections in red, while the right panel shows the resulting network after removing the corresponding unit and weights. In the SOC/SOH architecture this procedure is applied at the level of complete LSTM hidden channels and the associated MLP rows.

## 6. Quantization

Quantization complements pruning by reducing the numerical precision of the remaining weights while keeping the network topology unchanged. In this work we apply post-training quantization to the SOC and SOH LSTM + MLP models so that the recurrent weights are stored as 8-bit integers with associated floating-point scales, enabling compact storage and efficient integer arithmetic on the microcontroller. An 8-bit integer can encode $2^8 = 256$ distinct levels; a signed int8 representation uses the code book $\{-128, \ldots, 127\}$ so that positive and negative weights share a symmetric dynamic range around zero. An unsigned mapping $\{0, \ldots, 255\}$ would instead centre the quantizer around 127.5 and treat negative values asymmetrically, which is undesirable for zero-centred weight distributions. The factor 127 in the scale definition below therefore ensures that the largest-magnitude weight in a row maps close to $\pm 127$ while keeping zero exactly representable, following standard uniform int8 post-training quantization schemes [45, 46]. Conceptually, each FP32 weight $w$ is then mapped to an integer $q$ by a scalar quantizer and reconstructed approxi-

mately as

$$q = \mathcal{Q}_s(w), \tag{11}$$

$$\hat{w} = s\,q, \tag{12}$$

where $s > 0$ denotes a scale factor and $\mathcal{Q}_s : \mathbb{R} \to \{-128, \ldots, 127\}$ is a uniform mid-tread quantizer that rounds $w/s$ to the nearest integer and saturates the result to the closed interval $[-128, 127]$ whenever necessary.

### *6.1. Quantization Method*

In practice a per-row scale $s$ is chosen such that the maximum absolute weight in a row vector $w^{(r)} = (w_h)_h$ of a weight matrix $W$ (either $W_{\text{ih}}$ or $W_{\text{hh}}$) maps close to the int8 limits, for example

$$s = \frac{\max_h |w_h|}{127}, \tag{13}$$

and all weights in that row are quantized according to (11). The resulting integer weights and their FP32 scales are then consumed by the gate computation in (14), while biases and activations remain in floating point.

### *6.2. Quantization Details*

Per-row symmetric quantization is used for $W_{\text{ih}}$ and $W_{\text{hh}}$ so that each gate row receives its own floating scaling factor. Here $x$ denotes the current input feature vector, $h$ the previous hidden state, $W_{\text{ih}}^{\text{int8}}$ and $W_{\text{hh}}^{\text{int8}}$ the quantized input-to-gate and hidden-to-gate weight matrices, $s_{\text{ih}}$ and $s_{\text{hh}}$ the corresponding per-row scale vectors, and $b_{\text{fp32}}$ the FP32 bias of the considered gate. Biases and activations remain FP32, ensuring that the STM32 implementation reproduces the PyTorch activations gate by gate. A built-in streaming comparison validates the conversion and typically yields MAE differences below $10^{-4}$. The gate computation used in firmware is

$$g = (xW_{\text{ih}}^{\text{int8}}) \cdot s_{\text{ih}} + (hW_{\text{hh}}^{\text{int8}}) \cdot s_{\text{hh}} + b_{\text{fp32}}. \tag{14}$$

The INT8 representation reduces the recurrent weight memory footprint by roughly a factor of four compared with FP32 storage, which substantially eases

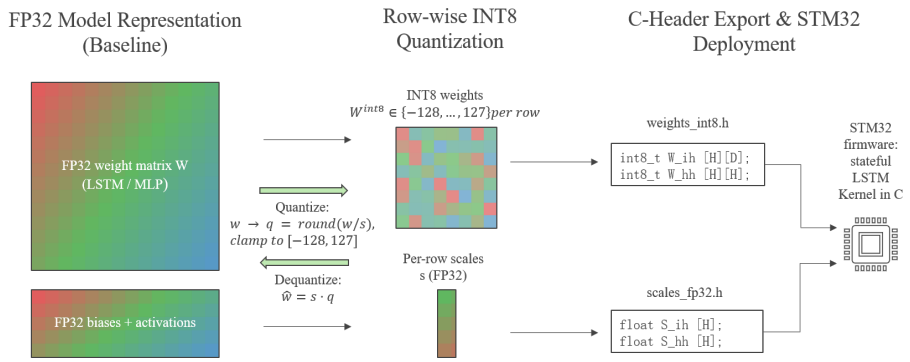


Figure 7: Illustration of the FP32-to-INT8 quantization scheme and embedded deployment. The left panel shows the baseline FP32 weight matrix and FP32 biases/activations. In the centre, per-row symmetric uniform quantization maps each row to an INT8 codebook $\{-128, \ldots, 127\}$ together with a floating-point scale, following (11)–(14). The resulting integer weights and scale vectors are exported as C headers and consumed by the stateful LSTM kernel on the STM32H753ZI, which reconstructs the effective weights during on-device inference.

the memory pressure when hosting the estimators on the STM32H753ZI MCU of the in-house BMS.

### *6.3. Export for STM32*

The exported headers integrate into the STM32 firmware project, which implements separate stateful LSTM + MLP models for SOC and for SOH as well as UART-based telemetry in C. A small driver library provides build files and flashing scripts so that identical measurement binaries can be rebuilt on other machines. This explicit tool chain, from training through pruning and quantization to firmware, avoids hidden compiler optimisations and makes the resource measurements in Section 7 reproducible.

## 7. Benchmark Methodology

This section describes how we benchmark the SOC and SOH models on both the host PC and an STM32 Nucleo-144 development board with an STM32H753ZI MCU [47, 26]. The microcontroller provides 2 MB of on-chip Flash and 1 MB of

SRAM, which constitute the budget for firmware code, model parameters, and runtime buffers. Starting from the trained LSTM + MLP models, we generate three embedded variants (full-precision baseline, pruned model, and quantized model) and evaluate them on the *LFP DoE Cycle Ageing Dataset*. The benchmark combines long-horizon streaming simulations with hardware-in-the-loop measurements and is fully scripted to ensure reproducible accuracy and resource evaluations.

On the host side, the complete reference ageing sequence is replayed in Python using the same streaming inference functions that are later exported to C. The SOC and SOH estimators are evaluated in continuous time without windowing, and error metrics (MAE, RMSE, $R^2$), error histograms, and parity plots are computed from the resulting trajectories. These simulations provide a high-precision reference for the embedded experiments and are used to assess how pruning and quantization affect the dynamics of the baseline models.

For the embedded benchmarks the base, pruned, and quantized SOC/SOH models are embedded into dedicated firmware projects for the STM32H753ZI. Because STM32Cube AI does not natively support stateful LSTM layers, the recurrent update equations, post-processing, and UART diagnostics are implemented manually in C. Identical stimulus streams are then injected via UART, allowing a direct comparison between host-side and on-device behaviour under the same operating conditions.

### *7.1. STM32 Measurements*

On the STM32 Nucleo-144 development board, dedicated SOC and SOH firmware images execute the embedded estimators while host-side scripts handle stimulus injection and UART logging. Recorded feature streams are pushed through the UART into the microcontroller, which runs the stateful LSTM + MLP in a tight loop and emits the corresponding SOC or SOH predictions together with timing diagnostics. Each sample produces two time stamps: a host-side latency in milliseconds that measures the delay between sending an input vector and receiving its estimate, and an on-device inference time in microseconds

that reflects only the execution time of the core LSTM and MLP kernels on the STM32 (derived from the DWT cycle counter). The average on-device inference time, combined with an empirically calibrated average board power, yields the energy per inference reported in Section 7.

### *7.2. Key Performance Indicators*

For each deployment we report embedded performance metrics in the form of average host latency, on-device inference time, memory usage (flash and SRAM for code, weights, and buffers), and energy per inference. The error metrics (MAE and RMSE) are analysed in the PC streaming section and in the corresponding histograms; Tables 1 and 2 therefore focus on STM32 resource and timing indicators. In these tables the latency entry denotes the mean end-to-end delay between transmitting a feature vector over UART and receiving the corresponding SOC/SOH estimate on the host, whereas the inference time isolates the execution of the LSTM + MLP kernels on the microcontroller and is measured via the DWT cycle counter. The RAM figures are obtained by summing the statically allocated data (.data and .bss) with the maximum stack usage observed during streaming, converted to kilobytes, and the energy per inference is approximated by multiplying the average on-device inference time with an empirically calibrated average board power of 0.5 W, yielding reproducible microjoule values without re-running the external power measurements.
For each deployment we report embedded performance metrics in the form of average host latency, on-device inference time, memory usage (flash and SRAM for code, weights, and buffers), and energy per inference. The error metrics (MAE and RMSE) are analysed in the PC streaming section and in the corresponding histograms; Tables 1 and 2 therefore focus on STM32 resource and timing indicators. In these tables the latency entry denotes the mean end-to-end delay between transmitting a feature vector over UART and receiving the corresponding SOC/SOH estimate on the host, whereas the inference time isolates the execution of the LSTM + MLP kernels on the microcontroller and is measured via the DWT cycle counter. The RAM figures are obtained by sum-

Table 1: Engineering KPIs for SOC models on STM32H753ZI (full-stream averages).

| Model | Latency [ms] | Inference [ms] | Flash [KB] | RAM [KB] | Energy [mJ] |
|---|---|---|---|---|---|
| Base FP32 | 12.70 | 1.40 | 105.32 | 4.93 | 0.70 |
| Pruned FP32 | 12.20 (-3.9%) | 0.80 (-42.9%) | 62.27 (-40.9%) | 4.03 (-18.3%) | 0.40 (-42.9%) |
| Quant INT8 | 18.48 (+45.5%) | 6.99 (+399%) | 52.48 (-50.2%) | 3.96 (-19.7%) | 3.49 (+399%) |

Table 2: Engineering KPIs for SOH models on STM32H753ZI (full-stream averages).

| Model | Latency [ms] | Inference [ms] | Flash [KB] | RAM [KB] | Energy [mJ] |
|---|---|---|---|---|---|
| Base FP32 | 34.88 | 22.73 | 335.00 | 8.69 | 11.37 |
| Pruned FP32 | 24.69 (-29.2%) | 12.72 (-44.0%) | 182.41 (-45.5%) | 6.96 (-19.9%) | 6.36 (-44.0%) |
| Quant INT8 | 41.23 (+18.2%) | 29.21 (+28.5%) | 138.00 (-58.8%) | 6.70 (-22.9%) | 14.60 (+28.5%) |

ming the statically allocated data (.data and .bss) with the maximum stack usage observed during streaming, converted to kilobytes, and the energy per inference is approximated by multiplying the average on-device inference time with an empirically calibrated average board power of 0.5 W, yielding reproducible millijoule values without re-running the external power measurements. An analogous set of measurements was recorded for the SOH firmware deployments. Table 2 lists the corresponding indicators.

The KPIs in Tables 1 and 2 highlight the trade-offs introduced by pruning and quantization. For SOC, pruning reduces the flash footprint by roughly 41 % and cuts both inference time and energy per inference by about 43 %, while keeping latency dominated by UART transfer. Quantization roughly halves the flash usage again, but the additional de-quantization operations and integer–float conversions inside the hand-written kernels increase the on-device inference time and energy by a factor of four compared with the dense baseline. The SOH deployments show the same qualitative behaviour: pruning saves about 46 % of flash and 44 % of inference energy, whereas the quantized variant attains the smallest flash footprint but incurs a moderate increase in computation time and

energy. These measurements demonstrate that pruning is the primary lever for improving energy efficiency on the STM32, while quantization is most beneficial when non-volatile memory is the limiting resource.

## 8. Results

In this section, we present a comprehensive evaluation of the proposed embedded AI pipeline. We analyse the performance of both the State-of-Charge (SOC) and State-of-Health (SOH) estimators, focusing on the trade-offs between the full-precision baseline, the pruned variant, and the quantized deployment. The results are derived from the continuous streaming replay of the high-resolution LFP DoE Cycle Ageing Dataset, ensuring that all transient effects and long-term drifts are captured.

### *8.1. Streaming Prediction Accuracy*

The primary metric for any battery state estimator is its ability to track the ground truth over extended periods without divergence. Figures 8 and 11 illustrate the streaming performance of the SOC and SOH models over the complete test duration. For the SOC estimator, the tracking is exceptionally tight across the entire dynamic range. The error residuals remain centered around zero, with only minor excursions during extreme current steps. Notably, the quantized model (blue trace) overlaps the full-precision baseline (green trace) so closely that they are often visually indistinguishable. This confirms that our layer-wise symmetric quantization strategy preserves the recurrent dynamics essential for accurate Coulomb counting and voltage relaxation modelling. The pruned model (red trace), while still highly accurate, exhibits slightly larger deviations in highly dynamic regions, a simplified behaviour expected from the significant reduction in hidden state dimensionality.

The SOH estimator faces a different challenge: it must infer slow capacity degradation from sparse check-up events and continuous monitoring. In the plotted trajectories the discrete capacity tests have been removed from the estimator input, while the remaining check-up blocks (OCV sweeps, HPPC pulses,

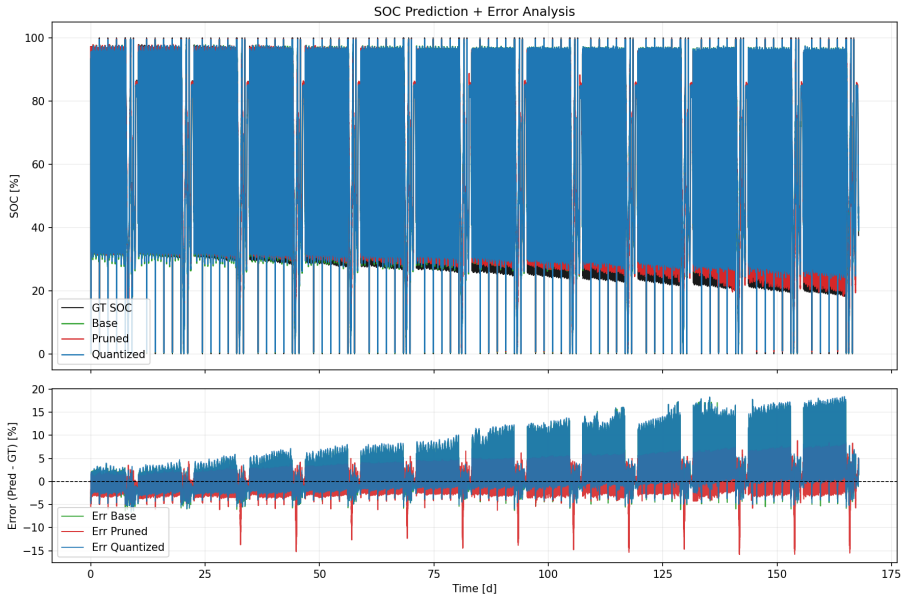


Figure 8: Streaming SOC dashboard on the full ageing trajectory. The black line represents the reference SOC, while the coloured lines show the predictions of the Base (green), Pruned (red), and Quantized (blue) models. The lower sub-panel displays the instantaneous prediction error.

and profile segments) are retained. The plateau-like sections of the ground-truth curve therefore correspond to these diagnostic intervals, during which ageing proceeds more slowly than under high-rate charge and discharge. A causal smoothing filter is applied to the raw SOH estimates to suppress high-frequency noise; the filter operates online and does not introduce a look-ahead. Specifically, we employ an exponential moving average (EMA) with a smoothing factor $\alpha = 10^{-6}$ combined with a downward-rate limiter that caps instantaneous capacity drops to $2 \times 10^{-8}$ per step, effectively rejecting unphysical jumps while tracking the monotonic degradation trend. After filtering, the pruned and quantized SOH models track the baseline with similar fidelity, capturing the long-term trend while exhibiting somewhat different levels of small-scale variability (Fig. 11).

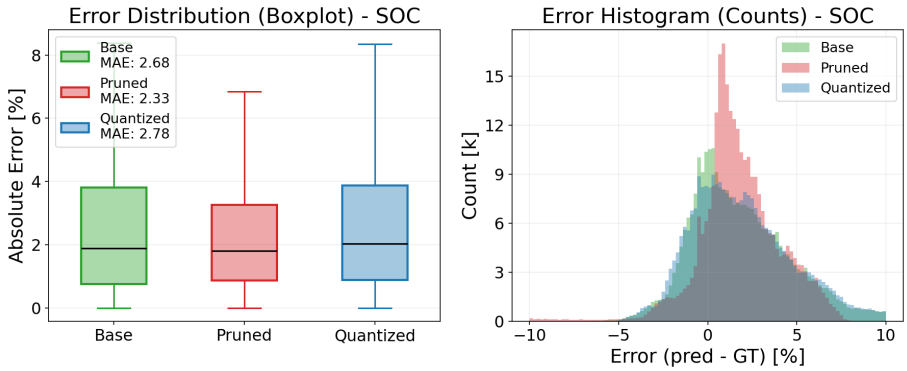


Figure 9: SOC Error Analysis: MAE distribution (left) and error histogram (right). The tight clustering around zero confirms unbiased estimation.

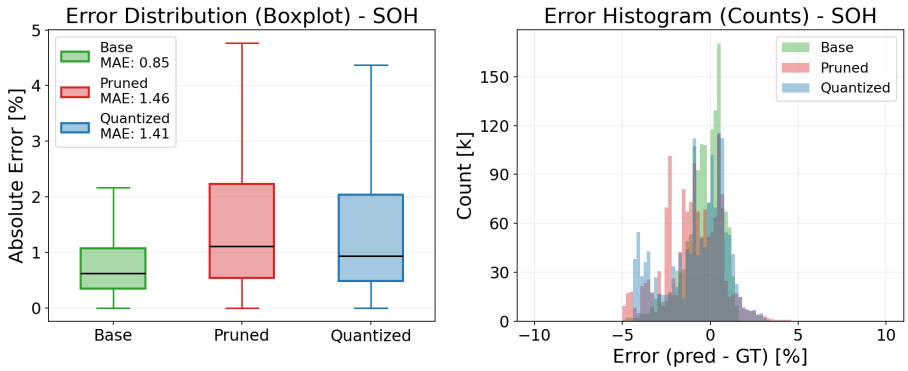


Figure 10: SOH Error Analysis: MAE distribution (left) and error histogram (right). The spread is naturally larger due to the difficulty of the task, yet the quantized model remains robust.

### *8.2. Error Distribution Analysis*

To quantify the reliability of the estimators, we analyse the statistical distribution of the prediction errors. Figures 9 and 10 present the Mean Absolute Error (MAE) boxplots and the error histograms for both tasks. The SOC error distribution is remarkably narrow and symmetric, indicating no systematic bias. The boxplots reveal that the pruned model attains the smallest median MAE and interquartile range with respect to the reference SOC, while the quantized model closely matches the full-precision baseline. Pruning thus provides a slight

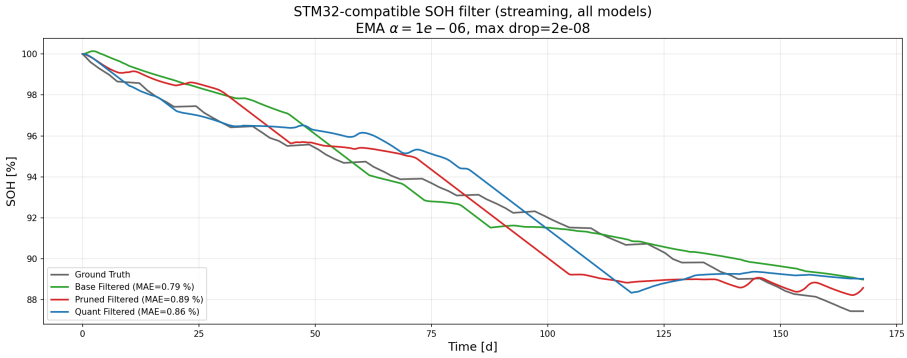


Figure 11: Streaming SOH dashboard with online filtering. The top panel shows the reference SOH and the outputs of the Base (green), Pruned (red), and Quantized (blue) models; the bottom panel depicts the corresponding prediction errors after causal smoothing.

accuracy gain at reduced runtime, whereas quantization preserves the baseline behaviour with a markedly smaller weight footprint, offering a complementary operating point when flash capacity is the primary constraint. For SOH, the error distribution is broader, which is inherent to the difficulty of estimating capacity fade from operational data. However, the histograms show that the bulk of the errors are still contained within a useful tolerance band (typically $\pm 2\%$), making the embedded estimator a viable tool for onboard diagnostics.

### *8.3. Transient Response and Zoomed Analysis*

Global metrics often hide local failures. To inspect the transient behaviour, we zoom into a high-dynamic pulse discharge phase (Fig. 12) and into a representative check-up sequence (Fig. 13).

In the pulse region (Fig. 12), the models must react to rapid current changes. The quantized model follows the ground truth transients with high fidelity, capturing the voltage recovery phases accurately. The pruned model, having fewer parameters to model the complex electrochemical relaxation, tends to smooth out these sharp features slightly. During the check-up phase (Fig. 13), where the cell undergoes full charge/discharge cycles, all models converge well, demonstrating stability over long integration horizons.

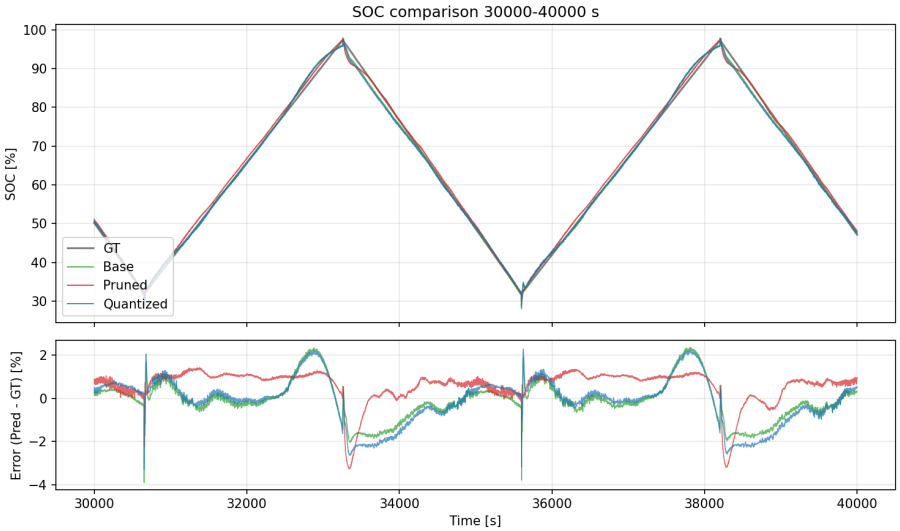


Figure 12: Detailed transient analysis of SOC streaming replay (pulse segment). Response to dynamic load pulses (30k–40k s).

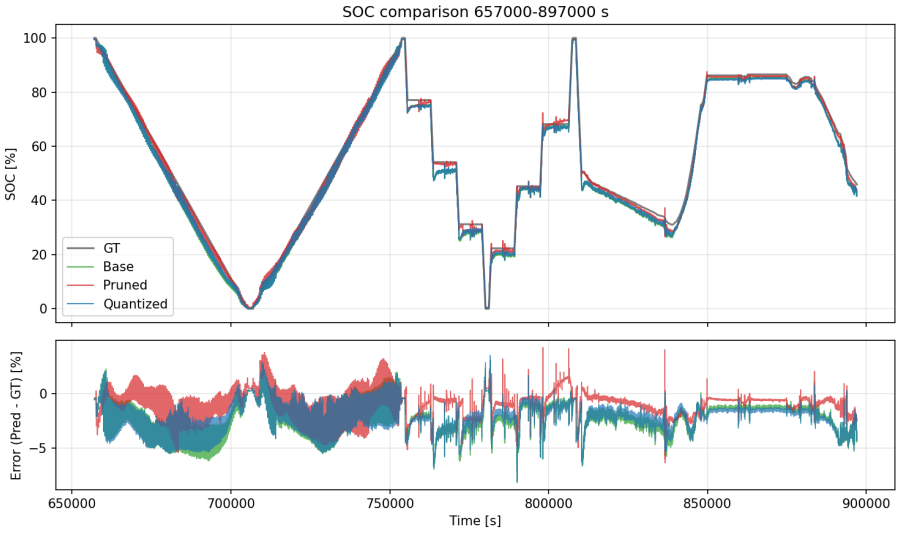


Figure 13: Detailed transient analysis of SOC streaming replay (check-up segment). Behaviour during a capacity check-up (657k–897k s).

### *8.4. Resource Efficiency and Latency*

Finally, we evaluate the cost of deployment. Figures 14 and 15 summarize the flash memory, RAM usage, and parameter counts for both SOC and SOH models and the corresponding latency distribution on the STM32H7 microcontroller. For all deployments the model complexity is first characterised on the workstation side by counting parameters in each variant (base, pruned, quantized). Assuming four bytes per weight for floating point representations and one byte per weight for INT8, this directly yields an estimate of the flash footprint attributable to the neural network weights alone. The second stage of the analysis uses the linker map files of the STM32CubeIDE projects to determine the actual binary size in flash, which also includes code, constant tables, and run-time support libraries. This explains why the measured flash usage in Figure 14 is systematically higher than the idealised "weights only" estimate, and why the gap is larger for the more complex SOH firmware that contains additional diagnostic and filtering logic. The RAM usage shown in the lower right panel of Figure 14 is obtained from on-device measurements of the SOH and SOC firmware and applied analogously to all STM32 deployments. At boot time the firmware reports the size of the statically allocated data and BSS sections, which correspond to persistent variables and network parameters stored in SRAM. During each benchmark inference the minimum value of the stack pointer is tracked, so that the maximum additional stack consumption of the call chain can be recovered. The total RAM requirement reported in the figure is the sum of static data and worst-case stack usage, i.e. an upper bound on the memory needed to execute one inference including the LSTM state updates and UART handling. This procedure captures the fact that pruning reduces both the number of parameters and the amount of intermediate state that must be kept in memory, whereas quantization primarily reduces the storage needed for the weights and activations, with a smaller impact on the stack frame size. The bar charts in Figure 14 clearly highlight the benefits of the optimisation pipeline. The quantized models (blue bars) achieve a dramatic reduction in Flash usage compared to the baseline (green), bringing the complex SOH model down to

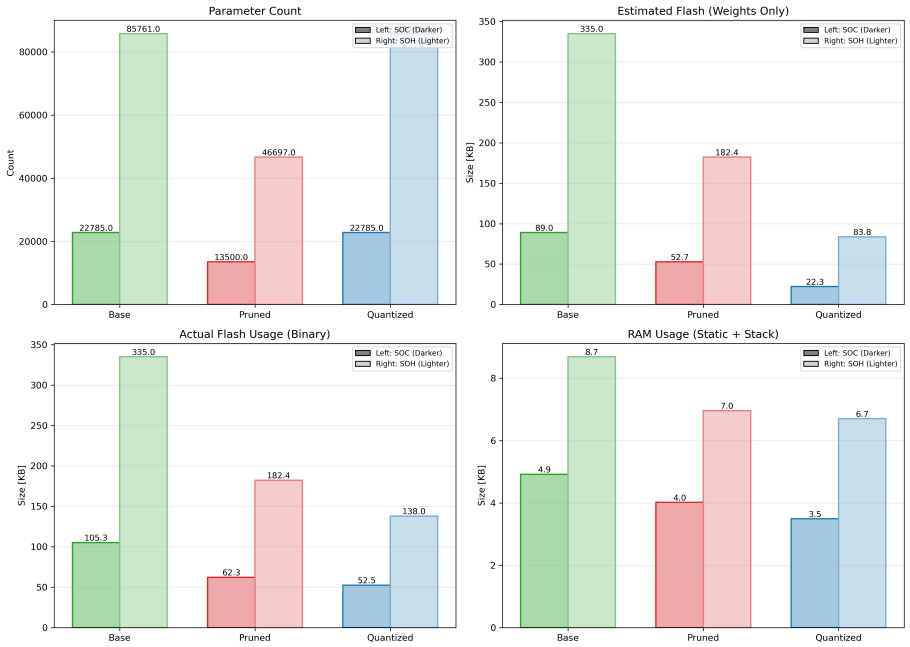


Figure 14: Resource landscape for SOC and SOH models. Comparison of parameter count, estimated Flash, actual Flash, and RAM usage. Dark bars denote SOC, light bars denote SOH.

a footprint that comfortably fits alongside the SOC estimator on a mid-range MCU. Pruning (red bars) is particularly effective at reducing the parameter count and RAM usage, which directly translates to shorter execution times.

Figure 15 reports the corresponding host-side latency distributions. The latency distributions in Figure 15 reveal a distinct hierarchy that is consistent with the model sizes. On the inference side, the SOC estimators execute in approximately 0.8 ms to 7 ms depending on the compression strategy, whereas the SOH variants require on the order of 12 ms to 30 ms per update. The fully quantized SOC deployment is the slowest in terms of pure inference time because the INT8 weights are unpacked inside a hand-written kernel, yet even here the computation remains well below 10 ms. When the host-side communication overhead (UART framing and Python processing) is included, the end-to-end latencies remain below roughly 20 ms for all SOC models and below 50 ms for the SOH

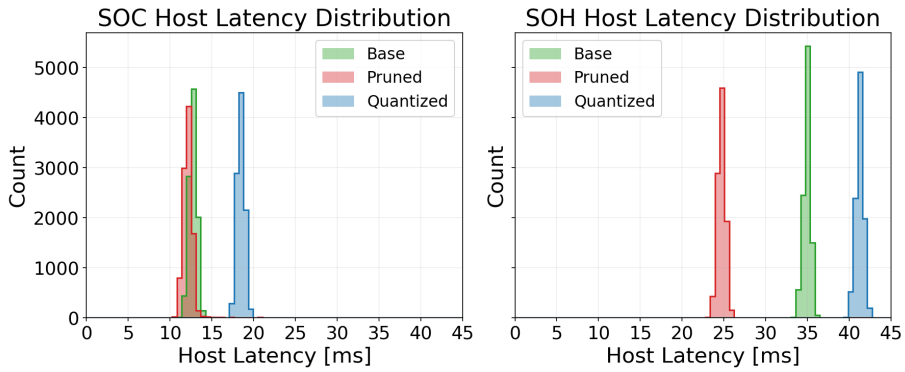


Figure 15: Latency distributions for SOC and SOH models on the STM32H7. Histograms are computed from repeated on-device measurements.

estimator, which is comfortably within the sampling periods of typical stationary storage BMS and low-rate diagnostic tasks. This analysis allows engineers to quantify the trade-off between compression, model complexity, and timing margin on a concrete MCU target.

### *8.5. Composite trade-off summary*

To make the multi-objective trade-offs easier to interpret at a glance, we report a simple composite utility score $U$ that aggregates accuracy and embedded resource indicators into a single, dimensionless index. For a given task (SOC or SOH) and a model variant $v$, we define

$$U_v = \frac{1}{4}\left(\frac{\mathrm{MAE}_v}{\mathrm{MAE}_{\mathrm{Base}}} + \frac{F_v}{F_{\mathrm{Base}}} + \frac{R_v}{R_{\mathrm{Base}}} + \frac{E_v}{E_{\mathrm{Base}}}\right), \tag{15}$$

where MAE is the streaming mean absolute error (in % of full scale), $F$ is the measured flash footprint, $R$ the measured total RAM usage (static + peak stack), and $E$ the measured energy per inference on the STM32. By construction $U_{\mathrm{Base}} = 1$; values $U < 1$ indicate that a variant improves, on average, across these four objectives. The score is intended as an illustrative summary only: deployments can prioritise objectives differently, which can be reflected by replacing the equal weighting in (15) with task-specific weights. Table 3 sum-

Table 3: At-a-glance effect of pruning and quantization relative to the corresponding Base model. Values are reported as SOC/SOH. ΔMAE is reported in percentage points (pp) of full scale (negative indicates improved accuracy); flash, RAM, and energy changes are relative deltas versus Base. The composite score $U$ follows (15) (lower is better). In this study pruning is the most effective lever for reducing energy, whereas quantization is the most effective lever for reducing flash.

| Variant | ΔMAE [pp] | ΔFlash [%] | ΔRAM [%] | ΔEnergy [%] | $U$ |
|---|---|---|---|---|---|
| Pruned | -0.35/+0.61 | -41/-46 | -18/-20 | -43/-44 | 0.71/0.91 |
| Quantized | +0.10/+0.56 | -50/-59 | -20/-23 | +399/+29 | 1.83/1.03 |

marises the resulting changes for pruning and quantization; values are reported as SOC/SOH.

## 9. Discussion

The experiments show that pruning and quantization affect the SOC and SOH estimators in different ways. For SOC, both the streaming plots and the error histograms indicate that compression can be applied without sacrificing estimation quality. The pruned SOC model achieves a slightly lower MAE and a narrower error distribution than the dense baseline, which is consistent with an over-parameterised 64-unit LSTM in which pruning primarily removes redundant units and acts as an additional regulariser. The quantized SOC model tracks both the baseline and the reference SOC almost indistinguishably in the long-horizon replay, with discrepancies that remain within the numerical noise introduced by the FP32 de-quantization.

For SOH the picture is more nuanced. The SOH targets evolve slowly, are observed reliably only during periodic capacity check-ups, and the online estimator operates on filtered signals. In this setting the 128-unit baseline uses its capacity more effectively, and both pruning and quantization introduce a noticeable but still bounded loss in accuracy. The error histograms exhibit broader tails for the compressed SOH models and do not reveal a clear winner between pruning and INT8. This behaviour is compatible with the higher intrinsic noise

level of the SOH regression task and suggests that aggressive compression should be combined with additional temporal filtering or architectural changes if very small error margins are required.

From an engineering perspective the three deployment variants represent distinct operating points. The base FP32 model offers the simplest tool flow but requires the largest flash footprint and the longest on-device execution time. Pruning reduces the recurrent state dimension and the associated MLP rows, shrinking the SOC flash usage from about 105 kB to 62 kB and shortening the on-device inference time from 1.40 ms to 0.80 ms (roughly 40 %). For the SOH estimator the corresponding numbers are 335 kB to 182 kB and 22.73 ms to 12.72 ms, respectively. Quantization, in turn, achieves the smallest flash and RAM footprint, with the SOC weight storage reduced by approximately a factor of four compared with FP32. On the STM32H7 this comes at the cost of additional de-quantization work inside the hand-written kernels: the quantized SOC model reaches 6.99 ms inference time, while the quantized SOH model trades part of the runtime gains of pruning for a further reduction in flash usage. Because the compressed deployments still meet real-time constraints with ample margin, the choice between pruned and quantized variants can be guided by the available flash budget and the strictness of the latency targets.

The distinction between system latency and inference time is particularly relevant for resource-constrained BMS. The UART-driven tests report end-to-end latencies between roughly 12 ms and 18 ms for the SOC variants and between 25 ms and 40 ms for the SOH models, values that are dominated by serial communication and host scheduling. The on-device inference times in Tables 1 and 2, by contrast, show that the SOC estimator requires less than 1.5 ms of compute time per sample, while the SOH estimator completes one update in approximately 23–30 ms. Typical sampling rates for pack-level monitoring in stationary storage and EV battery management systems lie in the single-digit to low double-digit hertz range [33, 3], so all investigated deployments satisfy real-time constraints with comfortable margin.

More generally, the reported measurements describe a multi-objective trade-

off space rather than a single "best" solution. In practical BMS deployments, accuracy, flash, RAM, and energy cannot typically be minimised simultaneously, so application-specific priorities and constraints must be specified explicitly. A pragmatic way to use our results is therefore constraint-driven: impose acceptable accuracy bounds (e.g., MAE/RMSE ceilings) together with hard memory budgets (flash/RAM), and then select the candidate that minimises energy per inference among the feasible models. If a single scalar ranking is desired, the normalised utility score $U$ presented in Table 3 offers a compact comparison. For the SOC estimator, the pruned variant achieves the most favourable score ($U = 0.71$), reflecting substantial gains in both efficiency and accuracy. In contrast, the quantized SOC model yields a score of $U = 1.83$ despite its minimal footprint, as the software-based de-quantization overhead significantly increases energy consumption. For the SOH task, both compressed models hover near the baseline utility ($U \approx 0.9$–$1.0$), indicating that the flash savings are roughly balanced by the slight degradation in accuracy and the overhead of the INT8 kernel. This underscores that while quantization is the most effective tool for minimising flash usage, pruning currently offers a more balanced profile for energy-constrained software implementations on the STM32H7.

Current limitations include the focus on a single, albeit rich, experimental campaign from the LFP DoE Cycle Ageing Dataset, the restriction to one cell chemistry, and the lack of a systematic study on measurement noise, sensor drift, or fault injection on the embedded platform.

## 10. Conclusion and Outlook

This work presents a complete and reproducible path from data-driven SOC and SOH estimators to an STM32-based battery management controller. Starting from LSTM + MLP models trained on the LFP DoE Cycle Ageing Dataset, we derive pruned and quantized variants and benchmark all deployments with respect to accuracy, timing, memory usage, and energy on an STM32H753ZI microcontroller.

For SOC, the experiments show that aggressive compression is possible without sacrificing estimation quality. The pruned SOC model reduces the recurrent state dimension while slightly improving MAE and narrowing the error distribution, and it lowers both the flash footprint and the energy per inference by roughly 40 % compared to the full-precision baseline. Quantization further shrinks the weight storage to about half of the original size and keeps the SOC accuracy within numerical distance of the full-precision model, but it increases the energy per inference because of the additional de-quantization work in the hand-written kernels. For SOH, pruning and quantization follow the same qualitative trends: flash savings on the order of 45–60 % and energy reductions of about 45 % for the pruned variant. The resulting accuracy/complexity trade-off is more modest because the underlying regression problem is noisier and driven by sparse capacity measurements.

The resource and latency measurements demonstrate that all investigated deployments meet typical real-time constraints for stationary storage applications by a comfortable margin, and that co-location of SOC and SOH estimators on a single STM32H7 is feasible even for the high-capacity SOH model. The methodology, including the quantization scheme, firmware integration, and benchmarking scripts, can be reused to compare alternative SOC/SOH estimators on equal footing.

## Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used ChatGPT, DeepL, and Grammarly in order to improve the language and style of the manuscript. After using these tools/services, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Future work will focus on transferring the embedded estimators from the evaluation board to a full battery management system with pack-level integration, where additional constraints such as communication bandwidth and safety

monitoring become critical. On the modelling side it will be important to extend the study to additional cell chemistries, temperature ranges, and loading regimes, and to compare the LSTM backbone against alternative architectures such as GRUs, Transformers, or hybrid observers that combine equivalent-circuit models with neural components [34, 48, 16, 18]. Another promising direction is to jointly tune pruning and quantization, combining the low memory footprint of INT8 weights with the computational efficiency of sparse connectivity. Specifically, quantizing an already pruned model could yield a synergistic effect, minimizing both Flash storage and execution time simultaneously. This would allow for even more compact deployments on ultra-low-power microcontrollers where both memory and compute cycles are scarce. These extensions will clarify how far the presented optimisation strategy generalises and where further algorithmic advances are required for next-generation resource-aware BMS.

## Data availability

The LFP DoE Cycle Ageing Dataset used in this work can be obtained from the DOI repository in [13]. The landing page provides licensing information and download instructions.

## Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author(s) used ChatGPT, DeepL, and Grammarly in order to improve the linguistic quality and style of the manuscript. After using these tools, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

## Data availability

The LFP DoE Cycle Ageing Dataset used in this work can be obtained from the DOI repository in [13]. The landing page provides licensing information and

download instructions.

## References


[1] J. Yao, J. Kowal, Towards a smarter battery management system: A critical review on deep learning-based state of charge estimation of lithium-ion batteries, Energy and AI 21 (2025) 100585. doi:10.1016/j.egyai.2025.100585.

[2] A. M. S. M. H. S. Attanayaka, J. P. Karunadasa, K. T. M. U. Hemapala, Department of Electrical Engineering, University of Moratuwa, Moratuwa, Sri Lanka, Estimation of state of charge for lithium-ion batteries - A Review, AIMS Energy 7 (2) (2019) 186–210. doi:10.3934/energy.2019.2.186.

[3] M. Berecibar, I. Gandiaga, I. Villarreal, N. Omar, J. Van Mierlo, P. Van Den Bossche, Critical review of state of health estimation methods of Li-ion batteries for real applications, Renewable and Sustainable Energy Reviews 56 (2016) 572–587. doi:10.1016/j.rser.2015.11.042.

[4] P. Shrivastava, P. A. Naidu, S. Sharma, B. K. Panigrahi, A. Garg, Review on technological advancement of lithium-ion battery states estimation methods for electric vehicle applications, Journal of Energy Storage 64 (2023) 107159. doi:10.1016/j.est.2023.107159.

[5] LEAP-RE, MG FARM Smart stand-alone micro-grids as a solution for agriculture farms electrification, https://www.leap-re.eu/mg-farm/.

[6] Microenergy, MG-FARM Powering Sustainable Agriculture with Smart Micro-Grids, https://mg-farm.microenergy-consulting.com/.

[7] A. Aqachtoul, K. Karam, A. Elamrani, Y. Alidrissi, M. Najib, A. Rochd, M. Bakhouya, MGFARM: An IoT and Edge-Driven Framework for Smart and Sustainable Agricultural Practices, in: Y. Mejdoub, A. Elamri, M. Kardouchi (Eds.), Connected Objects, Artificial Intelligence, Telecommunica-

tions and Electronics Engineering, Vol. 1584, Springer Nature Switzerland, Cham, 2026, pp. 419–425. doi:10.1007/978-3-032-01536-5_64.

[8] Technische Universität Berlin, MG Farm Smart microgrids as a solution for agriculture farms electrification, https://www.tu.berlin/eet/forschung/projekte/mg-farm.

[9] F. Rzepka, P. Hematty, M. Schmitz, J. Kowal, Neural Network Architecture for Determining the Aging of Stationary Storage Systems in Smart Grids, Energies 16 (17) (2023) 6103. doi:10.3390/en16176103.

[10] G. Crocioni, D. Pau, J.-M. Delorme, G. Gruosso, Li-Ion Batteries Parameter Estimation With Tiny Neural Networks Embedded on Intelligent IoT Microcontrollers, IEEE Access 8 (2020) 122135–122146. doi:10.1109/ACCESS.2020.3007046.

[11] D. P. Pau, A. Aniballi, Tiny Machine Learning Battery State-of-Charge Estimation Hardware Accelerated, Applied Sciences 14 (14) (2024) 6240. doi:10.3390/app14146240.

[12] M. A. Kamali, A. C. Caliwag, W. Lim, Novel SOH Estimation of Lithium-Ion Batteries for Real-Time Embedded Applications, IEEE Embedded Systems Letters 13 (4) (2021) 206–209. doi:10.1109/LES.2021.3078443.

[13] F. Rzepka, Design of Experiments for Cyclic Aging of LFP 18650 Cells: Impact of Charge/Discharge C-Rates, Depth of Discharge, and Temperature on Battery Degradation (Nov. 2025). doi:10.14279/DEPOSITONCE-24800.

[14] F. Guo, L. D. Couto, G. Mulder, K. Trad, G. Hu, O. Capron, K. Haghverdi, A systematic review of electrochemical model-based lithium-ion battery state estimation in battery management systems, Journal of Energy Storage 101 (2024) 113850. doi:10.1016/j.est.2024.113850.

[15] Y. Gao, K. Liu, C. Zhu, X. Zhang, D. Zhang, Co-Estimation of State-of-Charge and State-of- Health for Lithium-Ion Batteries Using an Enhanced

Electrochemical Model, IEEE Transactions on Industrial Electronics 69 (3) (2022) 2684–2696. doi:10.1109/TIE.2021.3066946.

[16] Y. Gao, G. L. Plett, G. Fan, X. Zhang, Enhanced state-of-charge estimation of LiFePO4 batteries using an augmented physics-based model, Journal of Power Sources 544 (2022) 231889. doi:10.1016/j.jpowsour.2022.231889.

[17] J. Shen, W. Ma, J. Xiong, X. Shu, Y. Zhang, Z. Chen, Y. Liu, Alternative combined co-estimation of state of charge and capacity for lithium-ion batteries in wide temperature scope, Energy 244 (2022) 123236. doi:10.1016/j.energy.2022.123236.

[18] C. Wang, N. Cui, Z. Cui, H. Yuan, C. Zhang, Fusion estimation of lithium-ion battery state of charge and state of health considering the effect of temperature, Journal of Energy Storage 53 (2022) 105075. doi:10.1016/j.est.2022.105075.

[19] W. Sun, B. Zhou, H. Xu, W. Yao, Y. Guo, Z. Yang, State of Charge Estimation of Sodium-Ion Batteries Based on N-BEATS Neural Network, in: 2023 7th Asian Conference on Artificial Intelligence Technology (ACAIT), IEEE, Jiaxing, China, 2023, pp. 1395–1401. doi:10.1109/ACAIT60137.2023.10528411.

[20] W. Sun, H. Xu, B. Zhou, Y. Guo, Y. Tang, W. Yao, Z. Yang, State-of-charge estimation of sodium-ion batteries: A fusion deep learning approach, Journal of Energy Storage 91 (2024) 111527. doi:10.1016/j.est.2024.111527.

[21] S. Giazitzis, M. Sakwa, S. Leva, E. Ogliari, S. Badha, F. Rosetti, A Case Study of a Tiny Machine Learning Application for Battery State-of-Charge Estimation, Electronics 13 (10) (2024) 1964. doi:10.3390/electronics13101964.

[22] W. Li, N. Sengupta, P. Dechent, D. Howey, A. Annaswamy, D. U. Sauer, Online capacity estimation of lithium-ion batteries with deep long short-term memory networks, Journal of Power Sources 482 (2021) 228863. doi:10.1016/j.jpowsour.2020.228863.

[23] ONNX Runtime: High-performance inference engine, https://onnxruntime.ai/.

[24] NVIDIA TensorRT: High-Performance Deep Learning Inference, https://developer.nvidia.com/tensorrt.

[25] X-CUBE-AI: STM32Cube Expansion Package for Artificial Intelligence, https://www.st.com/en/embedded-software/x-cube-ai.html.

[26] M. Naguib, P. Kollmeyer, C. Vidal, J. Duque, O. Gross, A. Emadi, Microprocessor Execution Time and Memory Use for Battery State of Charge Estimation Algorithms, in: WCX SAE World Congress Experience, Detroit & Online, Michigan, United States, 2022, pp. 2022–01–0697. doi:10.4271/2022-01-0697.

[27] J. Schulze, N. Strassenburg, T. Rabl, PQ Bench: Benchmarking Pruning and Quantization Techniques, in: Proceedings of the Workshop on Data Management for End-to-End Machine Learning, ACM, Berlin Germany, 2025, pp. 1–6. doi:10.1145/3735654.3735940.

[28] JGNE, JGCFR18650-1800mAh-3.2V Product Specification.

[29] D. Kucevic, B. Tepe, S. Englberger, A. Parlikar, M. Mühlbauer, O. Bohlen, A. Jossen, H. Hesse, Standard battery energy storage system profiles: Analysis of various applications for stationary energy storage systems using a holistic simulation framework, Journal of Energy Storage 28 (2020) 101077. doi:10.1016/j.est.2019.101077.

[30] K. Siebertz, D. Van Bebber, T. Hochkirchen, Statistische Versuchsplanung, Springer Berlin Heidelberg, Berlin, Heidelberg, 2017. doi:10.1007/978-3-662-55743-3.

[31] P. J. Huber, E. M. Ronchetti, Robust Statistics, 1st Edition, Wiley Series in Probability and Statistics, Wiley, 2009. doi:10.1002/9780470434697.

[32] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, A. Müller, J. Nothman, G. Louppe, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, É. Duchesnay, Scikit-learn: Machine Learning in Python (2012). doi:10.48550/ARXIV.1201.0490.

[33] G. L. Plett, Battery Management Systems: Volume 1: Battery Modeling, Artech House Power Engineering and Power Electronics, Artech House, Boston London, 2015.

[34] Z. C. Lipton, J. Berkowitz, C. Elkan, A Critical Review of Recurrent Neural Networks for Sequence Learning (2015). doi:10.48550/ARXIV.1506.00019.

[35] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, PyTorch: An Imperative Style, High-Performance Deep Learning Library (Dec. 2019). arXiv:1912.01703, doi:10.48550/arXiv.1912.01703.

[36] I. Loshchilov, F. Hutter, Decoupled Weight Decay Regularization (2017). doi:10.48550/ARXIV.1711.05101.

[37] I. Loshchilov, F. Hutter, SGDR: Stochastic Gradient Descent with Warm Restarts (2016). doi:10.48550/ARXIV.1608.03983.

[38] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, H. Wu, Mixed Precision Training (2017). doi:10.48550/ARXIV.1710.03740.

[39] R. Pascanu, T. Mikolov, Y. Bengio, On the difficulty of training Recurrent Neural Networks (2012). doi:10.48550/ARXIV.1211.5063.

[40] S. Han, J. Pool, J. Tran, W. J. Dally, Learning both Weights and Connections for Efficient Neural Networks (2015). doi:10.48550/ARXIV.1506.02626.

[41] D. Blalock, J. J. G. Ortiz, J. Frankle, J. Guttag, What is the State of Neural Network Pruning? (2020). doi:10.48550/ARXIV.2003.03033.

[42] W. Wen, C. Wu, Y. Wang, Y. Chen, H. Li, Learning Structured Sparsity in Deep Neural Networks (2016). doi:10.48550/ARXIV.1608.03665.

[43] E. Lobacheva, N. Chirkova, A. Markovich, D. Vetrov, Structured Sparsification of Gated Recurrent Neural Networks, Proceedings of the AAAI Conference on Artificial Intelligence 34 (04) (2020) 4989–4996. doi:10.1609/aaai.v34i04.5938.

[44] S. Narang, E. Elsen, G. Diamos, S. Sengupta, Exploring Sparsity in Recurrent Neural Networks (2017). doi:10.48550/ARXIV.1704.05119.

[45] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, D. Kalenichenko, Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference (2017). doi:10.48550/ARXIV.1712.05877.

[46] R. Krishnamoorthi, Quantizing deep convolutional networks for efficient inference: A whitepaper (2018). doi:10.48550/ARXIV.1806.08342.

[47] STM32 Nucleo-144 boards (Dec. 2025).

[48] S. Singh, P. Budarapu, Deep machine learning approaches for battery health monitoring, Energy 300 (2024) 131540. doi:10.1016/j.energy.2024.131540.